Assignment 1: Self-Supervised Learning with SimSiam on STL-10

B125808026 TAI, WEI-HSUAN

Abstract

We re-implement SimSiam [2] with a ResNet-18 backbone and pretrain it on the 100k unlabeled images of STL-10. With only 1,000 labeled images, a linear probe on the frozen SimSiam encoder reaches $80.98 \pm 0.35\%$ test accuracy (200 epochs, 3 pretraining seeds), while the same network trained from scratch reaches only $47.74 \pm 1.17\%$. We also reproduce the complete collapse reported in the original paper: removing either the stop-gradient or the predictor drops the linear probe to exactly chance level (10.00%). As a new ablation, we remove one family of augmentations at a time and find that the model degrades most severely without cropping. We further conduct extended experiments to understand why cropping plays such an important role, including cropping without rescaling, random masking and random patch shuffling.

1. Introduction

Self-supervised learning aims to learn visual representations from unlabeled images so that downstream tasks need only a few labels. SimSiam [2] is one of the simplest methods of this kind. It trains a Siamese network to make the representations of two augmented views of an image agree, and uses no negative pairs, no large batches and no momentum encoder. Its central claim is that a stop-gradient operation together with a predictor head is enough to keep the network from collapsing to a constant output.

Because SimSiam has no negative pairs, the difference between the two augmented views is its only learning signal. The original paper studies the architecture and the loss in detail, but it does not ablate the augmentations that create this signal. In this report we re-implement SimSiam on STL-10 [3] and study both sides. Our contributions are:

- We show that SimSiam pretraining on 100k unlabeled images beats training from scratch by 33.2 points when only 1,000 labels are available (Sec. 5.2).
- We reproduce the complete collapse caused by removing the stop-gradient or the predictor (Sec. 5.3).
- We ablate the augmentations and show that without cropping, SimSiam does not collapse to a point but suffers dimensional collapse, which worsens with longer training (Sec. 5.4).
- We show that neither the loss value nor the output std detects this failure, while the effective rank and a properly scaled linear probe do (Secs. 5.1, 5.4 and 5.5).

2. Related Work

Siamese self-supervised learning. Contrastive methods such as SimCLR [1] pull two views of the same image together and push views of different images apart. The negative pairs prevent the trivial constant solution. BYOL [4] removes the negatives and instead uses a predictor and a momentum encoder. SimSiam [2] shows that the momentum encoder is also unnecessary: a stop-gradient and a predictor alone are sufficient.

Role of augmentation. SimCLR [1] found that the combination of random cropping and color distortion is the most important augmentation pair for contrastive learning. In that setting the negatives provide a separate force against collapse. Whether the same holds for SimSiam, where augmentation is the only source of signal, has not been studied in the original paper.

Collapse. Complete collapse maps every input to the same vector. Dimensional collapse [6] is a milder failure: the embeddings stay spread out but occupy only a low-dimensional subspace. The output std monitored in SimSiam detects the former but not the latter. We therefore also measure the effective rank [7] of the embeddings.

3. Method

3.1. SimSiam

Given an image x, two random augmentations produce views x_1 and x_2. An encoder f, consisting of a backbone and a projection MLP, maps each view to $z_i = f(x_i)$, and a prediction MLP h maps it

to $p_i = h(z_i)$. With the negative cosine similarity $\mathcal{D}(p, z) = -\frac{p}{\|p\|_2} \cdot \frac{z}{\|z\|_2}$, the symmetric loss is

$$\mathcal{L} = \tfrac{1}{2}\mathcal{D}(p_1, \mathrm{sg}(z_2)) + \tfrac{1}{2}\mathcal{D}(p_2, \mathrm{sg}(z_1)), \tag{1}$$

where $\mathrm{sg}(\cdot)$ is the stop-gradient. The minimum of $\mathcal{L}$ is -1.

We use a ResNet-18 backbone [5] with its fc layer removed (512-d output), a 3-layer projection MLP ($512 \to 2048 \to 2048 \to 2048$, BN on every layer) and a bottleneck prediction MLP ($2048 \to 512 \to 2048$).

3.2. Ablations

Architecture. Following Sec. 4.1–4.2 of the original paper, we (a) remove the stop-gradient so that gradients flow through both branches, and (b) replace the predictor h with the identity.

Augmentation. The original paper's EM-like hypothesis (Sec. 5) views SimSiam as approximately solving

$$\min_{\theta} \mathbb{E}_{x,\mathcal{T}} \big\| \mathcal{F}_\theta(\mathcal{T}(x)) - \eta_x \big\|^2, \tag{2}$$

where η_x is the expected representation of x over augmentations $\mathcal{T}$. If the augmentations are too weak, this expectation becomes trivial. To test this, we remove one family of augmentations at a time: no_color drops color jitter and grayscale, and no_crop drops RandomResizedCrop, so that both views show the full image and differ only in flip and color.

3.3. Measuring Collapse

Besides the std of the ℓ_2-normalized output, which approaches $1/\sqrt{d}$ for outputs spread over the unit sphere and 0 under complete collapse, we measure the effective rank (erank) [7]. Let σ_k be the singular values of the centered, ℓ_2-normalized features of the 8,000 test images. Then

$$\mathrm{erank} = \exp\Big(-\sum_k p_k \log p_k\Big), \quad p_k = \frac{\sigma_k}{\sum_j \sigma_j}. \tag{3}$$

We focus on the 512-d backbone feature, which is the input to the linear probe.

4. Experimental Setup

Dataset. We use STL-10 [3] at 96×96. Pretraining uses the 100k unlabeled images. Classifiers train on the official fold 0 (1,000 images). Validation uses 500 held-out, label-balanced training images, and testing uses all 8,000 test images.

Pretraining. We use SGD (momentum 0.9, weight decay $5{\times}10^{-4}$) with batch size 512 and lr $= 0.03{\times}512/256$, with a 10-epoch warmup followed by cosine decay. The augmentation follows the original paper: RandomResizedCrop (scale 0.2–1), horizontal flip, color jitter $(0.4, 0.4, 0.4, 0.1)$ with p=0.8, and grayscale with p=0.2. A kNN monitor (k=200, τ=0.1) uses fold 0 as the memory bank and evaluates on the validation images after every epoch.

Evaluation. The linear probe trains a linear layer on the frozen backbone with SGD (lr 30, no weight decay) for 50 epochs. The BN probe adds a parameter-free BN before the linear layer, with lr chosen on validation. The from-scratch baseline (SFT) trains the whole ResNet-18 with the pretraining optimizer settings for 50 epochs. Every reported test accuracy uses the checkpoint with the best validation accuracy.

Statistics. For the 200-epoch comparisons we pretrain with seeds 123/456/789, and linear-probe each model with 5 finetuning seeds (12/34/56/78/90). The pretraining seed is the unit of replication: we report the mean ± std over the 3 per-model means and compare settings with two-sided Welch t-tests. When a setting has only one pretraining run, its ± reflects finetuning noise only, and we say so.

Implementation. All experiments ran on a single NVIDIA GeForce RTX 5070 Ti (16 GB) with CUDA 12.8, Python 3.10.21, PyTorch 2.8.0 and torchvision 0.23.0 (Fig. 1). One pretraining epoch takes about 45 s; a linear probe or SFT run takes 1–2 min. The code extends the course starter code: model.py (SimSiam, with stop_grad and predictor flags), data.py (two-view augmentation with an aug option), pretrain.py, linear_probe.py, sft.py and utils.py, plus analysis scripts for plotting, effective rank and statistical tests.

5. Experiments

Absolute numbers are not comparable to the original paper, which uses ImageNet, ResNet-50 and more labels, so we compare only qualitative trends.

5.1. SimSiam Pretraining

Figure 2 shows a 100-epoch pretraining run. The loss falls quickly from -0.15 to about -0.78 during the 10-epoch warmup and ends at -0.848, while the kNN accuracy rises steadily to 73.4%. The std of the normalized projector output ends at 0.02195, close to $1/\sqrt{2048} = 0.02210$, so the model has not collapsed.

```
(mai1) josh@josh-ubuntu:HW1$ python -c "import sys,torch,torchvision; print(sys.version); \
print(torch.__version__, torchvision.__version__, torch.version.cuda, \
torch.cuda.get_device_name(0))"
3.10.21 | packaged by conda-forge | (main, Aug 21 2026, 22:42:31) [GCC 14.4.0]
2.8.0+cu128 0.23.0+cu128 12.8 NVIDIA GeForce RTX 5070 Ti
(mai1) josh@josh-ubuntu:HW1$ nvidia-smi
Sun Oct 11 23:34:03 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 570.211.01             Driver Version: 570.211.01     CUDA Version: 12.8     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
|=========================================+========================+======================|
|   0  NVIDIA GeForce RTX 5070 Ti     On  |   00000000:01:00.0 Off |                  N/A |
|  0%   35C    P8             19W /  300W |      16MiB /  16303MiB |      0%      Default |
|                                         |                        |                  N/A |
+-----------------------------------------+------------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI              PID   Type   Process name                        GPU Memory |
|        ID   ID                                                               Usage      |
|=========================================================================================|
|    0   N/A  N/A            9452      G   /usr/lib/xorg/Xorg                        4MiB |
+-----------------------------------------------------------------------------------------+
```

Figure 1. Experimental environment.

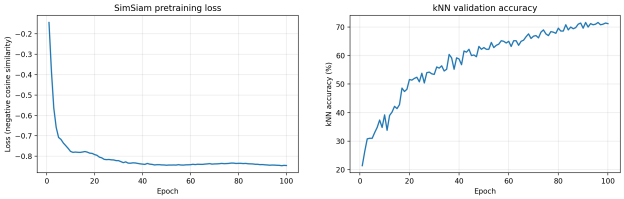


Figure 2. SimSiam pretraining for 100 epochs: training loss (left) and kNN validation accuracy (right).

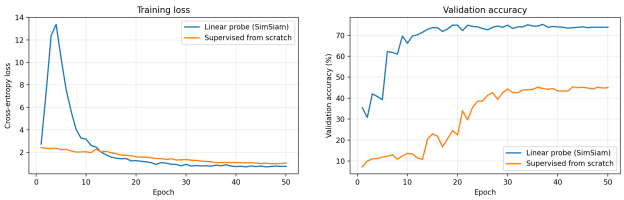


Figure 3. Linear probe on the frozen 100-epoch SimSiam encoder vs. SFT on the 1,000 fold-0 images: training loss (left) and validation accuracy (right).

Between epochs 50 and 80 the loss stays flat and even rises slightly ($-0.843 \rightarrow -0.837$), while the kNN accuracy climbs from 60.6% to about 70%. The SimSiam loss only measures agreement between two views, so its value says little about how useful the features are. We return to this point in Sec. 5.4.

5.2. Classification with and without Pretraining

Both classifiers use the same ResNet-18 and the same 1,000 labeled images (Fig. 3, Tab. 1). The linear probe trains only a 512×10 layer on frozen features. It passes 70% validation accuracy around epoch 10 and reaches a much lower training loss. SFT stays near chance for the first 13 epochs and plateaus around 45%. After 200 epochs of pretraining, the linear probe beats SFT by 33.2 points (95% CI 31.8–34.7, Welch t=59.2, p=2.4×10^{-8}).

Method	Pretrain	Test acc. (%)	n
SFT from scratch	–	47.74±1.17	5 ft. seeds
Linear probe	100 ep	75.80	1 run
Linear probe	200 ep	80.98±0.35	3 pt. seeds
Linear probe	1000 ep	86.74±0.23	1 run†

Table 1. Test accuracy with 1,000 labeled images. †One pretraining run, so ± covers finetuning seeds only.

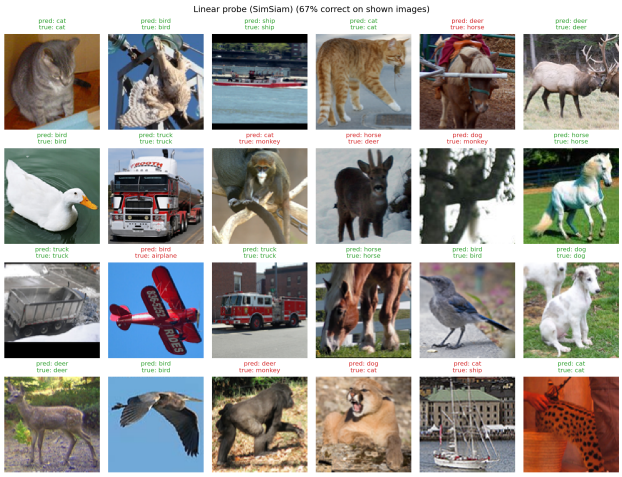


Figure 4. Linear probe predictions on random test images (green: correct, red: wrong). Errors concentrate among animal classes.

The gap is not caused by undertrained baselines. Training SFT longer saturates at about 49–50% (100 epochs: 49.62%; 200 epochs: 49.01%, with training loss 0.24, so it overfits), and a 200-epoch linear probe adds only +0.8 points over 50 epochs. SimSiam therefore learns from the 100k unlabeled images what 1,000 labels alone cannot teach.

Error analysis. In Fig. 4 and the confusion matrix (included in the code submission), the errors are mostly confusions between visually similar animals: cat↔dog, dog↔horse, monkey→cat/dog and deer↔horse. Vehicles (car, ship, truck, airplane) reach 86–93% per-class accuracy. Some mistakes come from context: a sailing ship in front of a building is labeled “cat”, and an airplane against a blue sky is labeled “bird”. Augmentation-invariant features describe global shape and texture well but can be misled by the background.

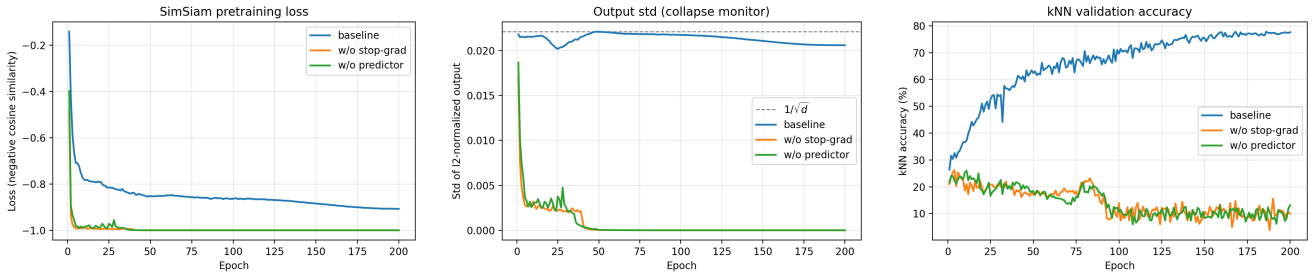


Figure 5. Ablating the stop-gradient and the predictor (200 epochs): training loss, std of the ℓ_2-normalized output (dashed: $1/\sqrt{d}$) and kNN accuracy. Both ablations collapse.

Setting	Loss	Std	kNN	Lin. probe
Baseline	−0.908	0.021	77.6	81.35
w/o stop-grad	−1.000	0	10.0	10.00
w/o predictor	−1.000	0	13.2	10.00

Table 2. Architecture ablations after 200 epochs (final-epoch values; one run each). After 100 epochs, the two ablations reach 14.89% and 16.16% linear probe accuracy.

Setting	Loss	kNN	Lin. probe	BN probe
Baseline	−0.909	78.1	80.98±0.35	82.36†
w/o color	−0.958	44.9	50.36±1.17	51.18†
w/o crop	−0.995	19.3	17.50±2.19	24.77±1.54

Table 3. Augmentation ablation, 200 epochs, mean (±std) over 3 pretraining seeds. Loss and kNN are final-epoch values averaged over seeds. †Seed 123 only.

5.3. Ablation: Stop-Gradient and Predictor

Both variants reproduce the collapse reported in the original paper (Fig. 5, Tab. 2). Within a few epochs the loss reaches its minimum of −1 and the output std drops toward 0, so every image is mapped to the same vector. After 200 epochs the linear probe scores exactly 10.00% with training loss $\ln 10 = 2.303$, which is chance level for 10 classes. The original paper's 0.1% is the same result for ImageNet's 1000 classes. After 100 epochs the probes still reach about 15%, because the backbone has not fully collapsed yet.

The collapse happens in two stages: in the first 5 epochs the std falls sharply to about 0.003, and between epochs 40 and 100 it decays slowly to 0. The two ablations produce nearly the same curves, which matches the original paper's analysis. Without a predictor, Eq. (1) has the same gradient as $\mathcal{D}(z_1, z_2)$, so removing the predictor in effect also removes the stop-gradient. Neither the architecture nor BN prevents the trivial solution; the asymmetry from the stop-gradient and the predictor is required. Because collapse here is deterministic, one run per variant is enough.

5.4. Ablation: Augmentation

Lower loss, worse representation. Both variants reach a lower loss than the baseline, because the views are easier to align, yet their accuracy is far lower (Tab. 3). Removing color costs 30.6 points (p=1.8×10^{-4}). Removing crop costs 56.2 points even under the BN probe (p=1.3×10^{-4}), and no_crop is 25.6 points below no_color (p=3.7×10^{-5}). Cropping is therefore the more important augmentation. This agrees with SimCLR, but the drop is larger here: without negatives, a too-easy view pair leaves no other signal to learn from. no_crop is also the least stable setting: its std across pretraining seeds is about 6× the baseline's (F-test $p \approx 0.025$, n=3).

The standard probe fails on no_crop. With lr 30, the linear probe on no_crop features never gets below training loss $\ln 10$, whether trained for 50, 100 or 200 epochs. The no_crop features occupy a tiny subspace with very small variance, so the fixed lr is poorly scaled for them. The BN probe standardizes the features (validation selects lr 1.0, versus 0.03 for the baseline) and raises no_crop to 24.77% (+7.27, paired t=19.2, p=0.003). We therefore report the BN probe for no_crop; the features remain about 58 points below the baseline.

no_crop gets worse with more training. no_color improves from 100 to 200 epochs (43.9 → 50.4), but no_crop moves the other way. With 3 seeded 100-epoch runs and the BN probe, its accuracy drops from 38.17 ± 2.39 to 24.77 ± 1.54 (Welch t=8.18, df 3.4, p=0.0023; 95% CI of the change $[-18.3, -8.5]$). Its kNN curve never rises steadily, unlike the baseline's (Fig. 6, left).

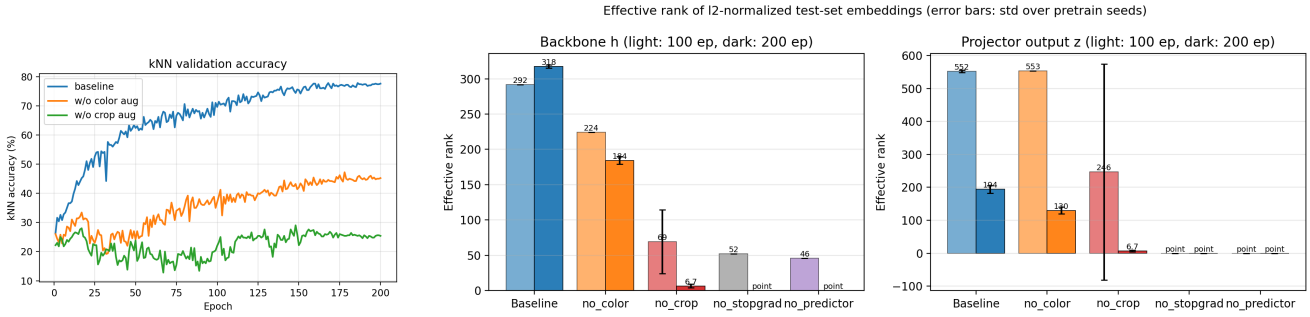


Figure 6. Left: kNN accuracy during augmentation-ablation pretraining (one representative 200-epoch run each). Right: effective rank of test-set embeddings for the backbone h and projector output z after 100 epochs (light) and 200 epochs (dark); error bars show std over pretraining seeds. "point" means all images map to one point.

Dimensional collapse, not complete collapse. The effective rank explains these results (Fig. 6, right). The 200-epoch no_stop-grad and no_predictor models are single points: their feature variance relative to the mean squared norm is 10^{-15} or smaller. no_crop is different. Its projector output z still spreads out (relative variance 0.93, nonzero std), but occupies only about 4–8 of 2048 dimensions, and its backbone erank is just 6.7 ± 2.8, versus 317.5 ± 2.4 for the baseline. This is dimensional collapse [6]. It worsens with training: for every seed, the backbone erank drops from 100 to 200 epochs ($53.3 \to 9.2$, $120.3 \to 7.4$, $34.2 \to 3.6$) together with the accuracy. It also explains why the standard probe failed: a few informative directions with tiny scale require a standardized head. In terms of Eq. (2), a crop-free pair differs only in color and flip. Averaging over such augmentations gives an η_x that a few color-invariant directions can match, and the objective keeps shrinking the representation onto those directions. Stop-gradient and the predictor prevent complete collapse, but not dimensional collapse.

5.5. Effect of Training Length

Longer schedules keep helping, with diminishing returns: 75.8% (100 epochs), 81.0% (200) and 86.7% (1000), matching the trend in Tab. 4 of the original paper (Fig. 7). Within the 1000-epoch run, the loss and kNN accuracy plateau from epoch 300 to 500 while the LR is still above half its peak. A saturation rule (less than 1 point gained per 100 epochs) would stop training at epoch 436, yet the linear probe accuracy keeps rising from 80% to 87% as the LR anneals. The plateau is therefore limited by the LR, not by the representation; real saturation happens only in the last 100 epochs (+0.07 points from epoch 950 to 1000). Snapshot accuracy also fluctuates: epoch 700 is 2.0 points below epoch 650, far more than the std across probe seeds (at most 0.6), so a single mid-schedule snapshot does not reliably represent "training up to that epoch".

6. Limitations

(i) Erank is not a quality measure across settings. For no_color, erank decreases from 100 to 200 epochs ($224 \to 184$) while accuracy increases ($43.9 \to 50.4$). Our evidence for no_crop is strong only because rank and accuracy fall together, and by large amounts. (ii) A falling std is not a sign of collapse either. The 1000-epoch baseline's std halves to 0.0100, yet its backbone erank (392) and accuracy (86.7%) are the highest of all our models. (iii) We use n=3 pretraining seeds, so the Welch df is only 2–4 and the CIs are wide. Even so, every effect above is many standard deviations in size.

7. Conclusion

SimSiam on STL-10 turns 100k unlabeled images into features that beat training from scratch by 33 points with only 1,000 labels. We confirmed that removing the stop-gradient or the predictor causes complete collapse. Our augmentation ablation reveals a second failure mode: when the views are too similar (no cropping), SimSiam keeps a nonzero std and a lower loss, yet its features collapse onto a few dimensions and degrade with more training. Neither the loss value nor the output std detects this; the effective rank and a properly scaled linear probe do.

References

[1] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In International Conference on Machine Learning (ICML), pages 1597–1607, 2020. 1

[2] Xinlei Chen and Kaiming He. Exploring simple Siamese representation learning. In IEEE/CVF Conference on

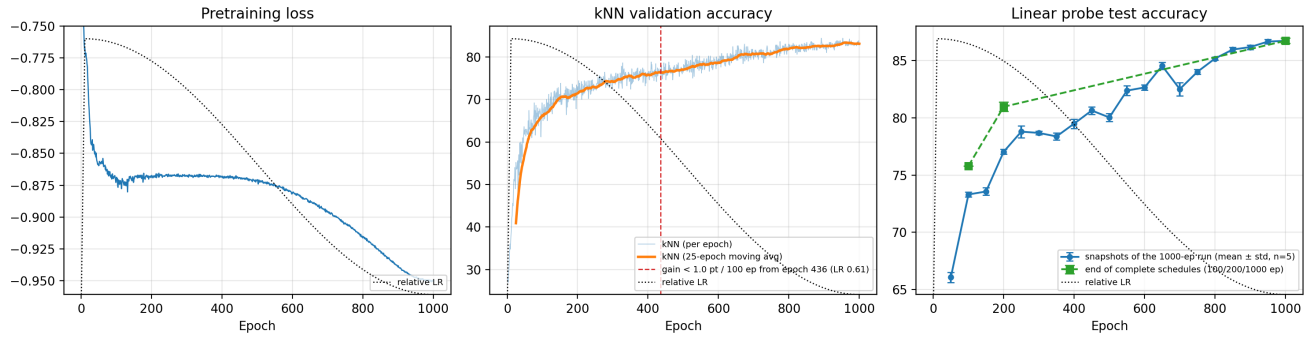


Figure 7. A 1000-epoch baseline run (dotted: relative LR). Right: linear probes on snapshots taken every 50 epochs (mean ± std over 5 probe seeds; green: final accuracy of complete 100/200/1000-epoch schedules).

Computer Vision and Pattern Recognition (CVPR), pages 15750–15758, 2021. 1

[3] Adam Coates, Andrew Ng, and Honglak Lee. An analysis of single-layer networks in unsupervised feature learning. In International Conference on Artificial Intelligence and Statistics (AISTATS), pages 215–223, 2011. 1, 2

[4] Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre H. Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Daniel Guo, Mohammad Gheshlaghi Azar, Bilal Piot, Koray Kavukcuoglu, Rémi Munos, and Michal Valko. Bootstrap your own latent: A new approach to self-supervised learning. In Advances in Neural Information Processing Systems (NeurIPS), 2020. 1

[5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pages 770–778, 2016. 2

[6] Li Jing, Pascal Vincent, Yann LeCun, and Yuandong Tian. Understanding dimensional collapse in contrastive self-supervised learning. In International Conference on Learning Representations (ICLR), 2022. 1, 5

[7] Olivier Roy and Martin Vetterli. The effective rank: A measure of effective dimensionality. In European Signal Processing Conference (EUSIPCO), pages 606–610, 2007. 1, 2